Choose an Appropriate Library

Explore the criteria for selecting an appropriate Python library to consume REST APIs effectively. Understand the differences between low-level and high-level HTTP libraries, and discover why the requests library suits most needs for both simple and configurable API consumption.

Before we start creating a client to consume REST services, we need to select the Python library that we're going to use.

First of all, we need to define what we need the library to do. Then, after looking at the available Python libraries, we'll be able to select the best match for our requirements. So, let's first specify the selection criteria.

Python

Our criteria

There are many libraries in every language that can be used to access a se[illegible] ranging from low-level TCP access-based libraries at one end to very high-level libraries at the other end that abstract out all the connection-related details. So,

Ask

let's define our requirements and conditions for choosing a library. What we need is a library with the following features:

1. It should abstract out the low-level connection details.
2. It should allow us to directly access/use the lower HTTP layers if required.
3. If point 2 isn't possible, then the library should at least let us configure the layers such as connection, session, and so on.
4. It should create a new connection pool or should be able to use an existing connection pool.
5. It should work seamlessly or with minimal configuration with marshaling and unmarshaling libraries.

Popular libraries

There are many libraries in Python that help in accessing a service. Some libraries provide low-level APIs to work with services, while others have not only abstracted out connection-level details but also integrated mechanisms to make asynchronous calls to services. Some advanced libraries even allow us to build reusable objects for consuming web APIs using decorators.

The following are the most commonly used libraries:

- **http.client**: The Python `http` module defines the classes that implement the client side of the HTTP and HTTPS protocols. In most cases, the `HTTP` module isn't used directly and is used by the `urllib` module to handle URLs that use HTTP and HTTPS.
- **httplib2**: The `httplib2` library is a comprehensive HTTP client library that supports many features left out of other HTTP libraries, such as support for HTTP and HTTPS, authentication, caching, redirects, compression, and lost update support. The module can handle any HTTP request method, r GET and POST.

Ask

- **urllib3**: The `urllib3` library is a powerful, user-friendly HTTP client for Python. `urllib3` brings many critical features that are missing from the standard Python libraries, such as thread safety, connection pooling, client-side TLS/SSL verification, file uploads with multipart encoding, helpers for retrying requests and dealing with HTTP redirects, support for gzip, deflate, and brotli encoding, etc.
- **requests**: The `requests` library is an elegant and simple HTTP library for Python. It allows us to send HTTP/1.1 requests quite easily. There's no need to manually add query strings to our URLs or to form-encode PUT & POST data. Keep-alive and HTTP connection pooling are 100% automatic, thanks to `urllib3`. Other features include sessions with cookie persistence, browser-style TLS/SSL verification, Basic/Digest authentication, automatic content decompression and decoding, Unicode response bodies, and more.
- **httpx**: The `httpx` library not only gives us all the standard features of the `requests` library but also sync and async APIs, HTTP/1.1 and HTTP/2 support, the ability to make direct requests to WSGI or ASGI applications, strict timeout everywhere, full type annotation, and so on. Please note that even though the `httpx` library follows the `requests` library's API layout, it doesn't use the `requests` library itself.
- **uplink**: The `uplink` client is a declarative HTTP client for Python. Inspired by RetroFit, it works with `requests`, `aiohttp`, and `twisted`. The `uplink` client turns our HTTP API into a Python class, i.e., it allows us to build reusable objects for consuming web APIs. It provides advanced features such as using decorators and type hints, support for JSON, URL-encoded, and multipart request bodies and file uploads, URL parameter replacement, request headers, query parameter support, non-blocking I/O support, easy deserialization/serialization, the ability to define custom converters for personalized objects, support for marshmallow schemas and handling collections, and a lot more.

Ask

We can see from the above list the range of available Python libraries from the low-level `http.client` library to the very high-level `uplink` library.

Selection criteria

Our choice

Based on our criteria, the `requests` library is the best option. On one hand, it lets us issue a `GET` command with just the URL of the service, and on the other hand, it provides many options to configure how we connect to the service.

Hence, we will be connecting to a server using the `requests` library.

Mark As Completed

Ask